

How Mabat loads, serves and saves typed aggregates

Mabat (Hebrew for “view”) loads nested, typed data from PostgreSQL, MySQL and SQLite. You declare the **view** of the data you want as Rust structs; Mabat plans the batched queries that fill it, lets a DBA replace any of them without touching your code, serves the view as JSON or GraphQL, and saves it back — without a session, proxies or lazy loading.

Every claim is a rule. This document explains the architecture that the **MPA** specification (the Mabat Persistence Architecture, docs/mpa.md) states as numbered rules, the way JPA states the contract of Java persistence providers. Where a page says what Mabat does, it cites the rule — [MPA-WRITE-9](#) — so it can be checked against the specification, and the specification against the tests. The last page is drawn from the specification's machine-readable index, docs/mpa.json, at build time.

The question. An ORM maps rows to objects and then decides, while your code runs, which queries to send: it loads lazily, tracks changes through a session, and flushes them later. Mabat takes the other road. What shape of data does a view declare, which queries does it become, who may change them, how do they run concurrently, and how does a value get written back — and how does that compare, point for point, with an ORM?

The sources. The MPA specification v0.1 (docs/mpa.md) and its index (docs/mpa.json), the design document (docs/design.md), and the code of the mabat crates 0.1: mabat-derive (the derive), mabat-core (shapes, plans and SQL, with no database), mabat-sqlx (execution, overrides and writes), mabat-graphql and mabat-cli. The SQL on these pages is what the planner and the writer generate, abridged only where marked.

Regenerating. docs/architecture/overview/mabat-visio.py draws the ten figures as one Visio file (mabat-architecture.vsd) with the poster kit in docs/architecture/ and its gates; it also writes the SVGs this document references and an EMF per figure for Office. build.sh runs the gates, checks every reference, renders this PDF with Chrome and checks one PDF page per section. Edit the HTML or the script, never the SVGs or the PDF.

1 · The short answer — five stages from a struct to the database, the same shape the other way for writes, and what that changes compared with an ORM.

2 · A view is a shape — what a declaration says, what is decoded from the row, and what becomes a query of its own.

3 · From shape to queries — the plan of a view, its SQL, and why 20 tasks cost five queries rather than 181.

4 · Tuning without code — override files, the checks that prepare every query at startup, shadow mode, reloads and the command line tool.

5 · Shapes of a result — trees, shared values, graphs with cycles, and two ways to load recursion.

6 · Concurrency — one connection, or a pool of them on one snapshot, level by level.

7 · JSON and GraphQL — selections, the generated schema, and paging the elements of each parent in one query.

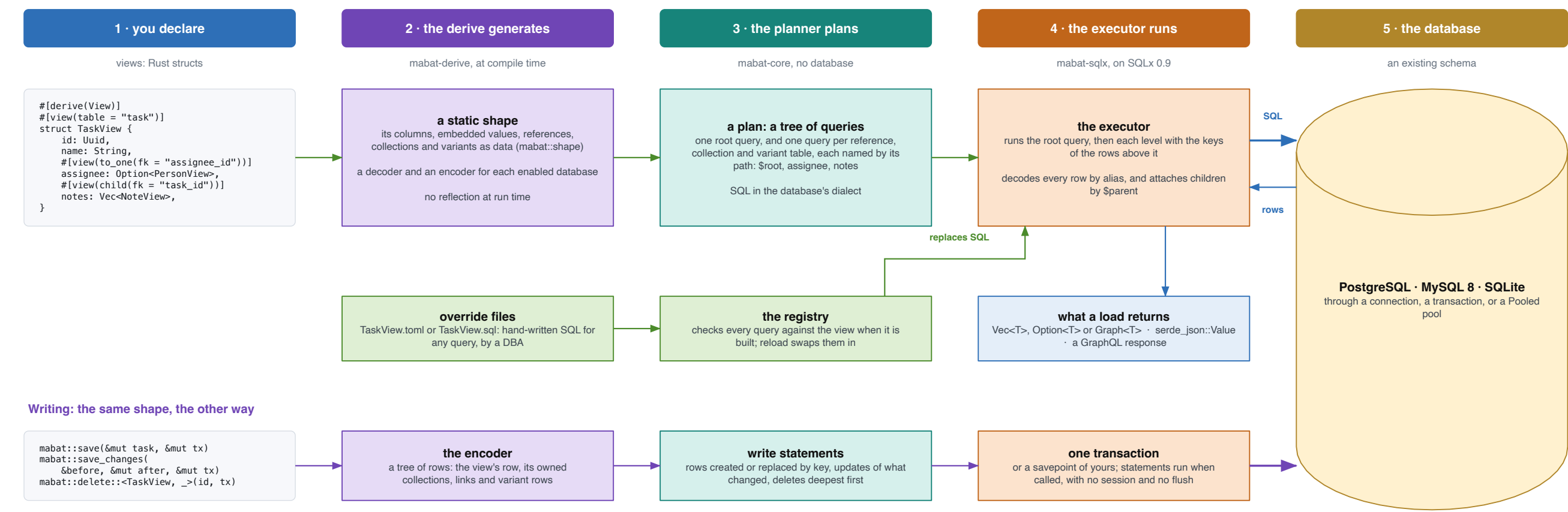
8 · Writing an aggregate — the tree of rows, the statements, and optimistic locking with a version column.

9 · Change tracking, against an ORM — an ORM's session, snapshots and proxies, beside two values compared by generated code.

10 · Three databases — what differs between PostgreSQL, MySQL and SQLite, and what does not.

11 · The contract — every capability against the areas of the specification, read from docs/mpa.json.

A view is a Rust struct that declares the shape of the data to load. The derive turns it into a static shape at compile time; the planner turns the shape into a tree of queries, one per relationship; the executor runs them level by level, each batched by the keys of the rows above, and decodes every row by its alias. A DBA can replace the SQL of any query, checked against the type when the application starts. Writes use the same shape the other way: an encoder builds a tree of rows, and the statements run when they are called.



an ORM · JPA and Hibernate, Diesel associations, SeaORM	Mabat
THE UNIT OF A READ an entity; what else loads depends on what the code touches (lazy loading) or on fetch joins chosen in code	THE UNIT OF A READ a declared view, loaded whole: the roots, and everything the view declares (MPA-CORE-2, MPA-NOT-4)
THE QUERIES decided at run time by navigation: N+1 queries, or joins that multiply collections, unless tuned in code	THE QUERIES fixed by the shape and printable before running: one per relationship, never one per row, never a join of two collections (MPA-PLAN-1)
WHO OWNS THE SQL the ORM; changing a query means changing and releasing code	WHO OWNS THE SQL generated, or replaced per query by a DBA and checked against the type at startup (MPA-OVR-5)
STATE AND WRITES a session: identity map, snapshots, dirty checking and a flush	STATE AND WRITES none: values are plain structs; save, save_changes and delete run when called (MPA-WRITE-1)

Declared, not discovered. A view says everything a load will fetch: its columns, embedded values, to-one references and to-many collections [MPA-CORE-1](#). The values it loads form an aggregate: the row, its embedded values, the rows of its owned collections and its variant tables [MPA-CORE-2](#). There is no lazy loading [MPA-NOT-4](#): a load never reaches the database again because code touched a field. The same table may have many views, each the shape one use of the data needs.

Generated at compile time. `#[derive(View)]` generates the shape as static data, and for each enabled database a decoder and an encoder; nothing is computed by reflection at run time [MPA-CORE-3](#). Views whose column types cannot be written, or serialized as JSON, still compile and fail only when used that way [MPA-WRITE-11](#) [MPA-JSON-2](#). `mabat-core`, which plans and renders SQL, has no database dependency, so planning is unit-tested without one.

Against an ORM. An ORM's unit of a read is an entity, and what else loads depends on what the code touches; its queries are decided at run time, and its state is a session. Mabat's queries are fixed by the shape and can be printed before they run [MPA-PLAN-5](#); its values are plain structs; and its writes run when called, in your transaction [MPA-WRITE-1](#). Page 9 compares the two on change tracking, point by point.

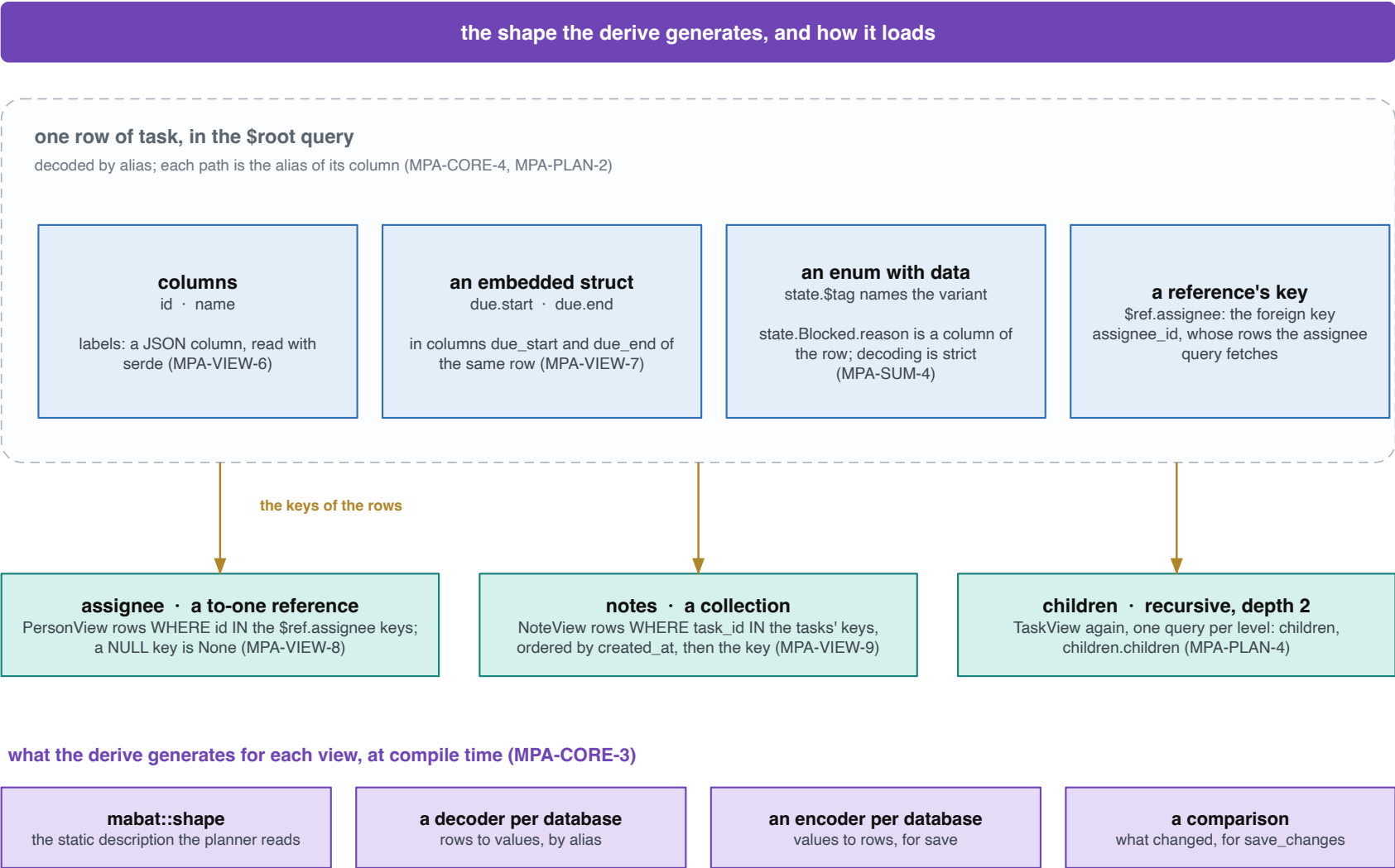
Everything a view declares is either decoded from the view's own row or loaded by a query of its own. Columns, JSON columns, embedded structs and enums stored in columns come from the row of the root query, each under an alias equal to its path. A to-one reference contributes only its foreign key to the row; the referenced rows, the elements of each collection and each level of a recursive collection are fetched by queries of their own, with the keys of the rows above.

the declaration

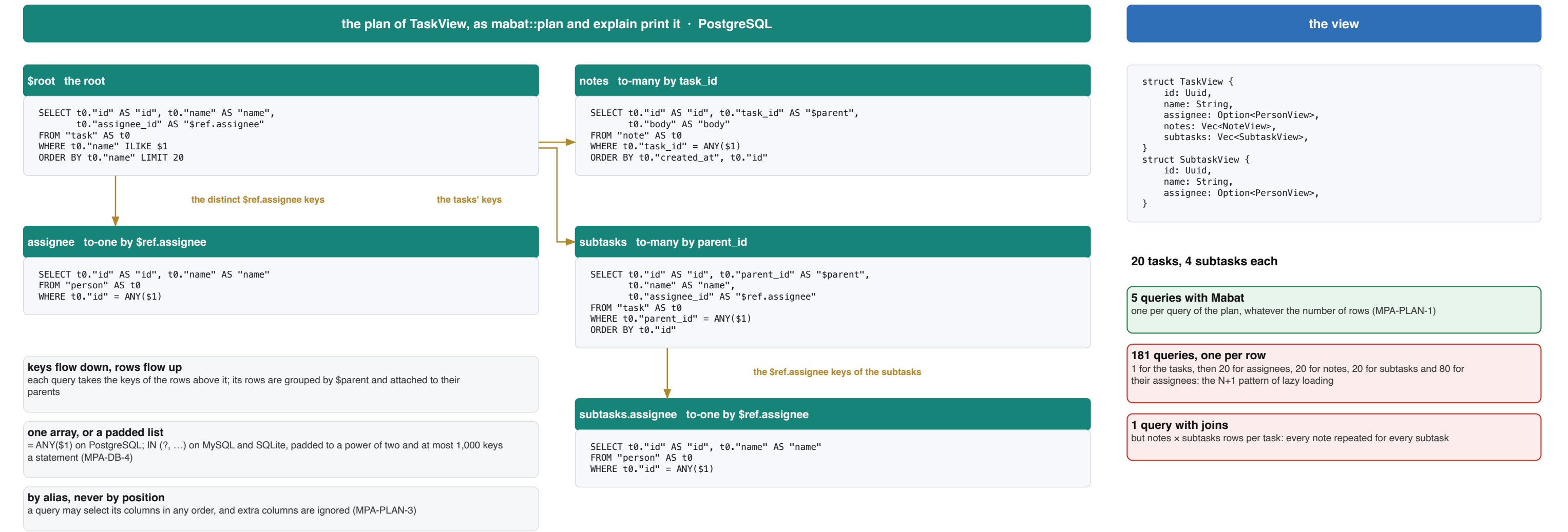
```
#[derive(View)]
#[view(table = "task")]
struct TaskView {
    id: Uuid,                // the key
    name: String,
    #[view(json)]
    labels: Vec<String>,
    #[view(embed(prefix = "due_"))]
    due: Window,             // due_start, due_end
    #[view(embed)]
    state: State,            // an enum with data
    #[view(to_one(fk = "assignee_id"))]
    assignee: Option<PersonView>,
    #[view(child(fk = "task_id", order_by = "created_at"))]
    notes: Vec<NoteView>,
    #[view(child(fk = "parent_id", depth = 2))]
    children: Vec<TaskView>,
}

#[derive(View)]
#[view(embedded)]
struct Window { start: NaiveDate, end: Option<NaiveDate> }

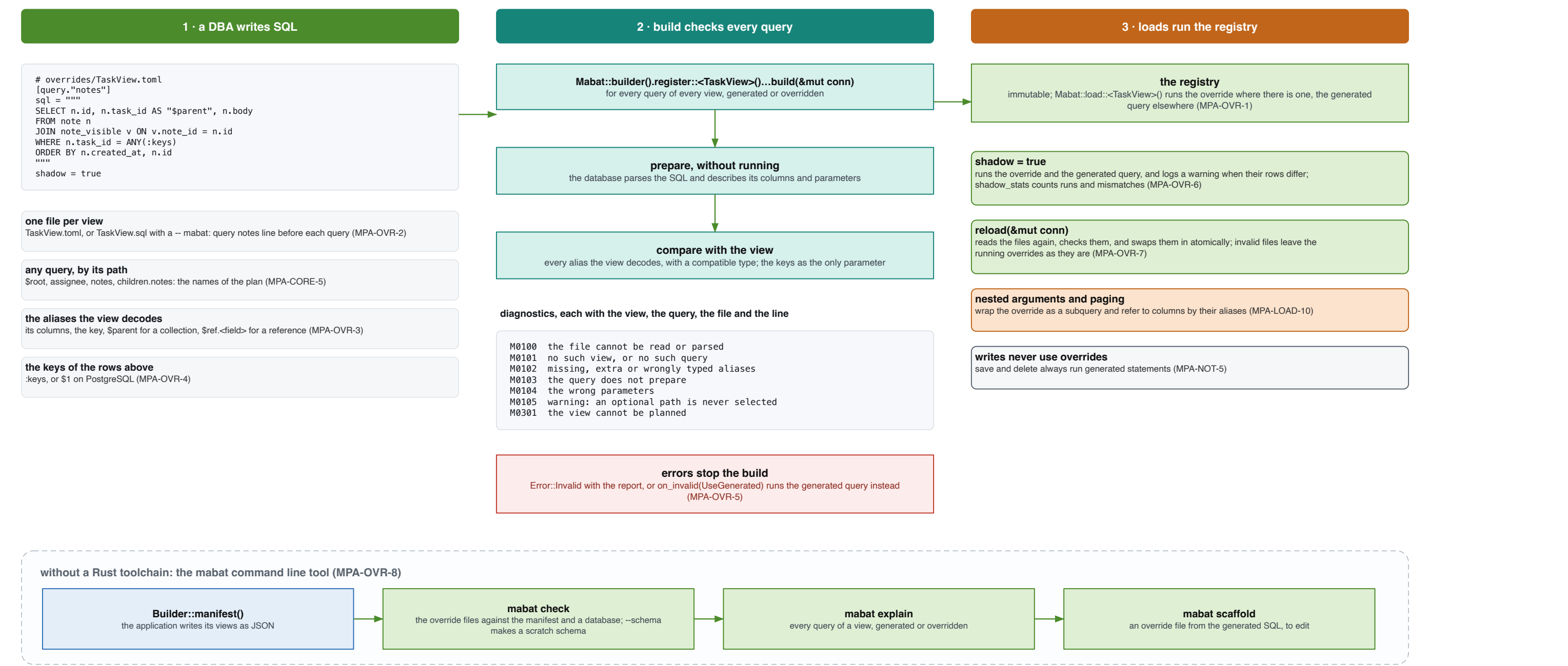
#[derive(View)]
#[view(tag = "state")]
enum State { Open, Blocked { reason: String }, Done }
```



The planner turns a shape into a tree of queries: one root query, and one per to-one reference, per collection and per variant table, at any depth, each batched by the keys of the rows above it. The number of queries depends on the shape, never on the number of rows, and no query joins two collections. The plan, and the SQL of each query, can be printed before anything runs.



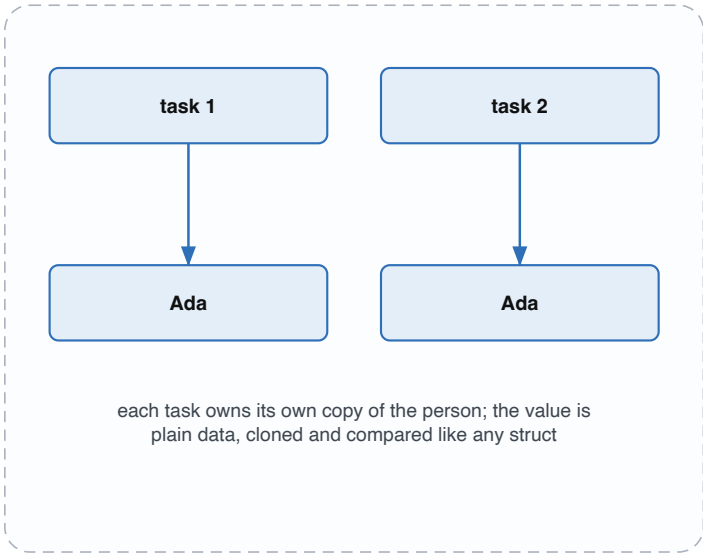
The SQL of any query of a view can be replaced by a file a DBA writes, with no change to the application. Because rows are decoded by alias, an override only has to select the aliases the view decodes and take the keys of the rows above. When the registry is built, every query — generated or overridden — is prepared on the database, without running, and its columns and parameters are compared with the view, so a wrong override stops the application at startup rather than corrupting a result later.



The field types of a view decide what a load returns. Plain fields make a tree of owned values, which can be cloned, compared and saved like any struct. `Arc<T>` shares one decoded value between everything that references the same row. `Ref<T>` makes the view a graph, held in an arena with each entity once, so cyclic data needs no depth limit. Recursive collections load a fixed number of levels, or every level in one recursive query.

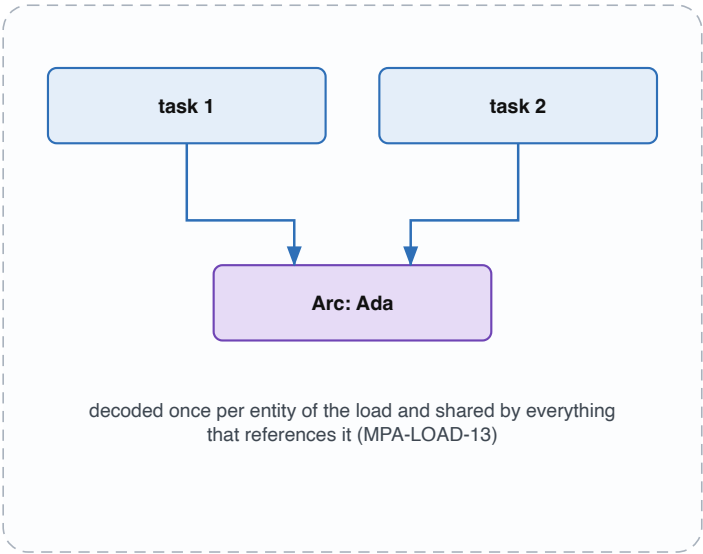
a tree

`Vec<T>`, `Option<T>`: the default



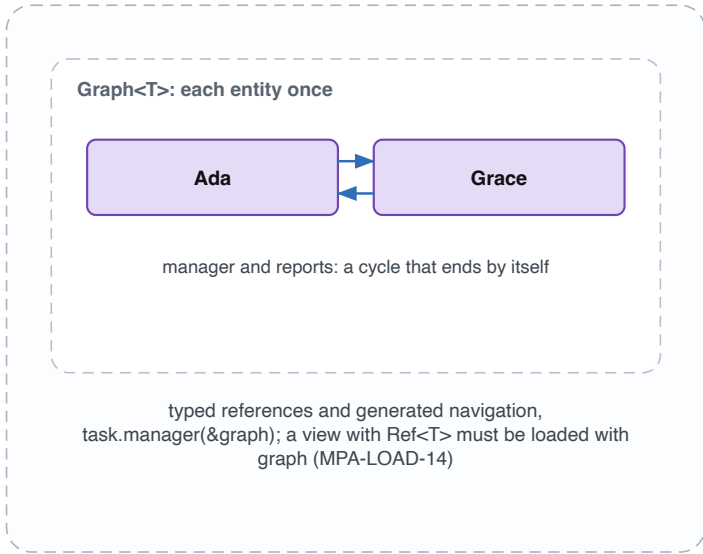
shared values

`Arc<T>`: decoded once per load



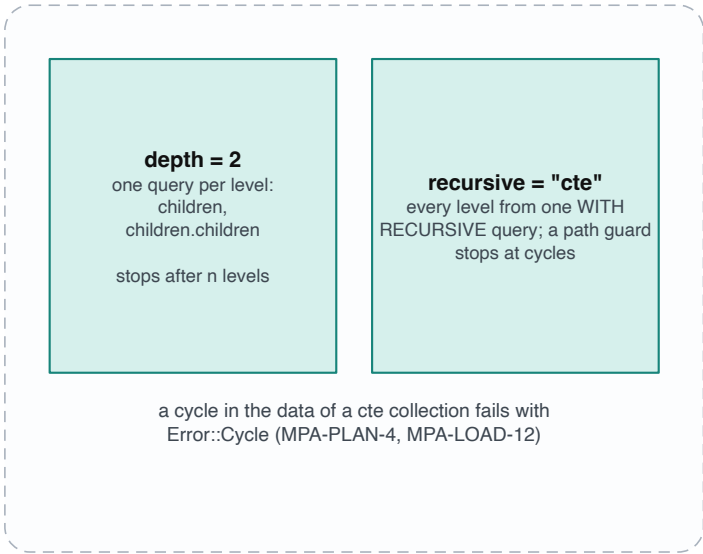
a graph

`Ref<T>`, loaded with `graph()`



recursion

depth = n, or recursive = "cte"



THE FIELD
`T`, `Option<T>`, `Vec<T>`, `BTreeMap<K, T>`

THE TERMINAL
all, one, optional, json

SAVING
save, save_changes, delete

THE FIELD
`Arc<T>`, `Option<Arc<T>>`, `Vec<Arc<T>>`

THE TERMINAL
all, one, optional, json

SAVING
save writes the referencing key only (MPA-WRITE-5)

THE FIELD
`Ref<T>`, `Option<Ref<T>>`, `Vec<Ref<T>>`

THE TERMINAL
graph, or json with a selection (MPA-JSON-5)

SAVING
save_graph, ordered by references (MPA-WRITE-14)

THE FIELD
`Vec<T>` of the view itself

THE TERMINAL
all, one, optional, json

SAVING
save writes every level it holds

Trees and shared values. By default a reference or element is an owned value: two tasks with the same assignee hold two equal copies. An `Arc<T>` reference or element is decoded once per entity of a load and shared by everything that references it [MPA-LOAD-13](#), which saves memory and decoding where the same rows recur. Collections are `Vec`, `BTreeMap` or `HashMap` of a view, of `Arc` or of `Ref` [MPA-VIEW-10](#); a list placed by an `index` column fails on a `NULL` or repeated index [MPA-VIEW-9](#).

Graphs. A view with `Ref<T>` fields is a graph and must be loaded with `graph`, else `Error::GraphRequired`. It returns a `Graph<T>` holding each entity once, with typed references and generated navigation methods such as `task.manager(&graph)`; cycles end by themselves [MPA-LOAD-14](#). A graph can be served as JSON only through a selection, whose finite depth unrolls it into a tree [MPA-JSON-5](#). A graph is saved with `save_graph`: every entity, each after the entities it references, with the optional references of a cycle set afterwards [MPA-WRITE-14](#) [MPA-WRITE-15](#).

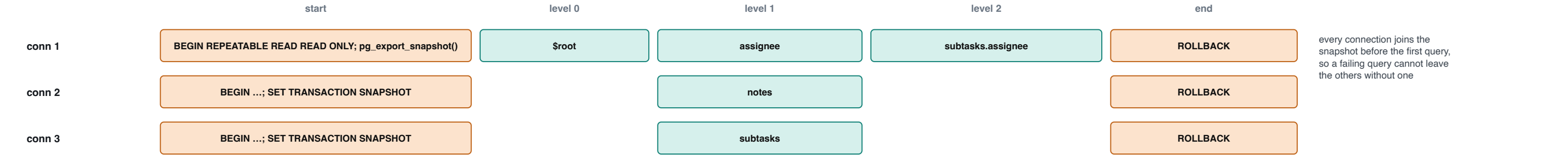
Recursion. A collection with `depth = n` runs its query again for each level, at most `n` levels; with `recursive = "cte"` every level comes from one `WITH RECURSIVE` query whose path guard stops at cycles, for a collection that contains its own view directly [MPA-PLAN-4](#). A cycle in the data of a cte collection fails with `Error::Cycle` [MPA-LOAD-12](#). A recursive view without either is a planning error: the depth of a tree must be known.

A load runs on whatever it is given. On a connection or a transaction, every query runs in turn and sees the transaction's own uncommitted writes. On a Pooled pool, the queries of each level run at the same time, each on a pooled connection held for that query only — and on PostgreSQL, all of them can share one snapshot, so the levels of a load see the same database even while others write.

one connection or transaction: every query in turn, all seeing the transaction's own writes (MPA-DB-5)



Pooled::snapshot(&pool, 3): each level's queries at the same time, on one snapshot · PostgreSQL (MPA-LOAD-11)



a connection

THE CONNECTION GIVEN
a connection, a transaction or a pooled connection: every query on it, in turn

CONNECTIONS
one, the caller's

WHAT THE LOAD SEES
the transaction, including its uncommitted writes

GRAPHS
in turn

Pooled::snapshot

THE CONNECTION GIVEN
Pooled::snapshot(&pool, n): PostgreSQL only — asking for it on MySQL or SQLite does not compile

CONNECTIONS
at most n, each held for one query; a load cannot deadlock on its own pool

WHAT THE LOAD SEES
one consistent snapshot for every query of the load

GRAPHS
in turn on one connection: which query reaches an entity first decides where its row comes from

Pooled::read_committed

THE CONNECTION GIVEN
Pooled::read_committed(&pool, n): any database; each query sees what is committed when it runs

CONNECTIONS
at most n, each held for one query

WHAT THE LOAD SEES
possibly a newer commit in a later level than in an earlier one

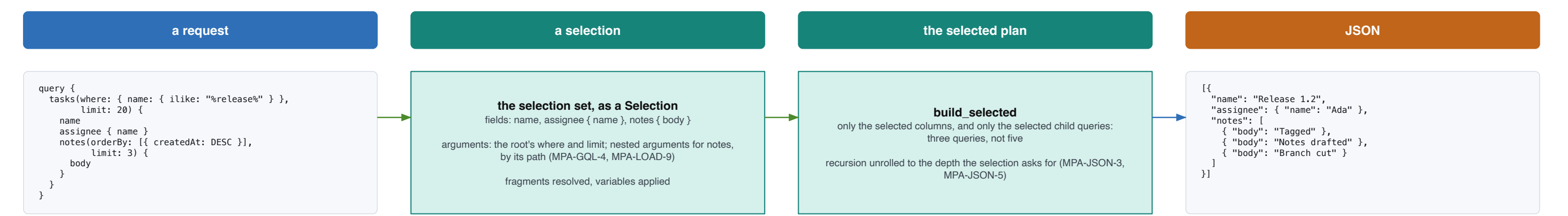
GRAPHS
in turn on one connection

The caller's connection. Wherever a load or write takes a connection, it accepts a connection, a `sqlx::Transaction`, a pooled connection or a Pooled pool, and every query of a load runs on the connection given, so it sees the uncommitted writes of its transaction [MPA-DB-5](#). That is the default because it is the one that is always correct. Queries are not pipelined on one connection [MPA-NOT-8](#): SQLx 0.9 has no pipelining API.

A pool, level by level. With a Pooled pool the child queries of each level run together, each on a connection held for that query only, so a load uses at most the `connections` of its Pooled and cannot deadlock on its own pool [MPA-LOAD-11](#). The keys of a level are collected from the rows above before its queries start. Pooled::snapshot begins REPEATABLE READ READ ONLY on one connection, exports its snapshot, and imports it on the others; it exists only for PostgreSQL, so asking for it on MySQL or SQLite does not compile.

Joining up front. Every connection imports the snapshot before the first query runs. If a query failed while others were still joining, its transaction could end under them; joining first removes that race. Pooled::read_committed works on any database and lets each query see what is committed when it runs. Graph loads run their queries one at a time on one connection, because which query reaches an entity first decides where its row comes from, and that should not vary from run to run.

A view can be loaded as JSON, whole or as a selection of its fields. A selection loads only what it names — only the selected columns, only the selected child queries — so a GraphQL request becomes one load of exactly the data it asks for. `mabat-graphql` generates the schema from the views, and turns each root field's selection set, with its arguments, into that load.



notes(limit: 3) is per task, in the notes query: ROW_NUMBER() OVER (PARTITION BY the parent)

```
SELECT * FROM (
  SELECT t0."id" AS "id", t0."task_id" AS "$parent",
    t0."body" AS "body",
    ROW_NUMBER() OVER (PARTITION BY t0."task_id"
      ORDER BY t0."created_at" DESC, t0."id") AS "$row"
  FROM "note" AS t0
  WHERE t0."task_id" = ANY($1)
) AS p WHERE p."$row" <= 3 ORDER BY p."$row"
```

the schema is generated from the views
mabat_graphql::schema(&pool).list::<TaskView>("tasks").by_key::<TaskView>("task"): objects for views, unions for enums with data, filters from the column types (MPA-GQL-1, MPA-GQL-2)

one load per root field
the whole selection set, every level batched: no resolver per field, so no N+1 (MPA-GQL-4)

the same, without GraphQL
load::<TaskView>().select(Selection::parse("name assignee { name }")).json(conn): for a REST endpoint (MPA-JSON-3)

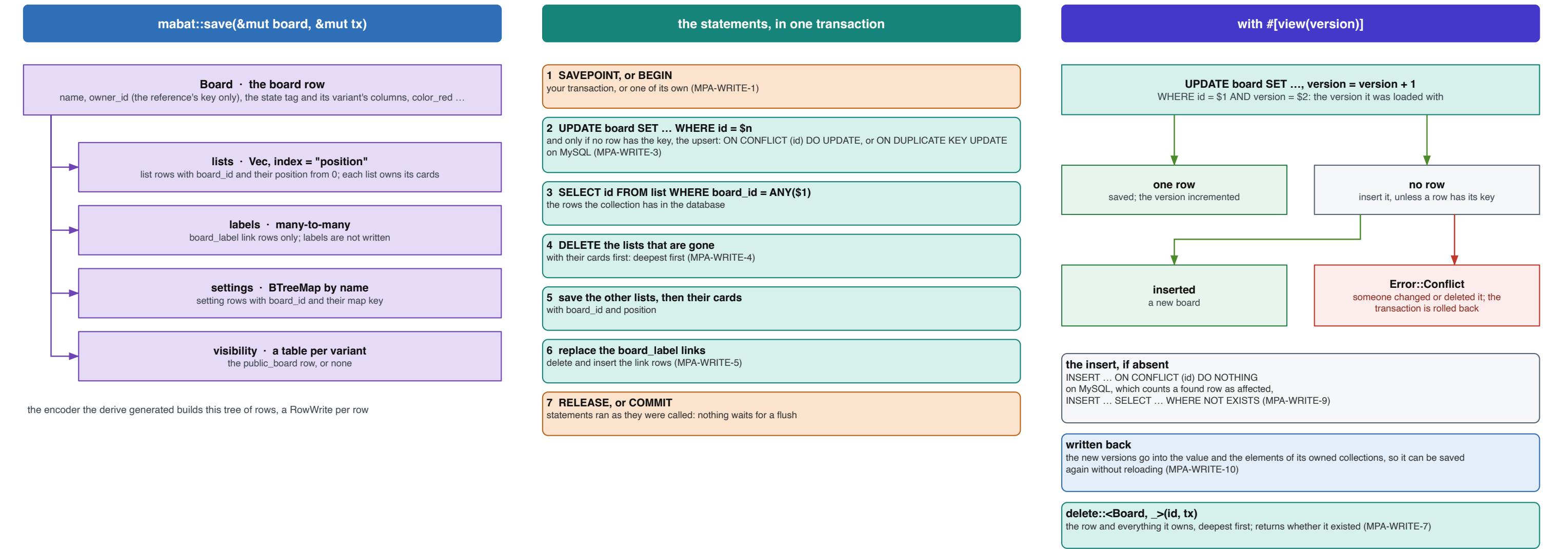
overrides apply
a registry's overrides serve selections and GraphQL too (MPA-JSON-6)

JSON. `load::<T>().json(conn)` returns `serde_json::Value` objects: columns through their type's `Serialize`, `json` columns as the JSON they hold, collections as arrays, maps as objects, references as objects or `null`, and enums as objects whose `__typename` names the variant [MPA-JSON-1](#). `select(Selection)` loads only the selected fields; `Selection::parse` reads GraphQL-like text [MPA-JSON-3](#). A view selected without fields loads its columns and embedded values, not its relationships [MPA-JSON-4](#). Overrides apply to selections [MPA-JSON-6](#).

A generated schema. `mabat_graphql::schema(&pool).list::<T>("tasks").by_key::<T>("task").finish()` builds an `async-graphql` dynamic schema from the views and every view they reach [MPA-GQL-1](#): objects for views and embedded structs, unions for enums with data, GraphQL enums for enums without, and scalars from the column types [MPA-GQL-2](#). List fields take `where`, `orderBy`, `limit` and `offset` [MPA-GQL-3](#). There are no mutations in 0.1 [MPA-NOT-7](#).

Arguments at every level. Each root field is one load of its selection set [MPA-GQL-4](#): no resolver per field, so no N+1. Nested arguments filter, order and page the elements of each parent inside the collection's one query — the filter follows the parent keys condition, `order_by` replaces the collection's order, and `limit` and `offset` apply per parent with `ROW_NUMBER() OVER (PARTITION BY ...)` [MPA-LOAD-9](#). Over an override, the arguments wrap it as a subquery and refer to columns by alias [MPA-LOAD-10](#).

The shape that loads an aggregate also saves it. `save` writes the view's row and makes each owned collection in the database equal to the value's; `delete` removes the row and everything it owns. Both run in your transaction, or a savepoint of it, and each statement runs when it is called: there is no session to flush. A `#[view(version)]` column makes concurrent writers fail instead of overwriting each other.



An ORM tracks changes **transparently**: the objects you load belong to a session that keeps a snapshot of each, replaces your references and collections with proxies and wrappers that load and record on their own, and at flush compares every object with its snapshot to decide what to write. Mabat tracks changes **explicitly**: you keep the value as it was loaded, change a copy, and `save_changes(&before, &mut after)` compares the two with code the derive generated, writing only the rows that differ — now, in your transaction.

an ORM · transparent: the session watches the objects

the session · persistence context, EntityManager

identity map

one managed object per row it has loaded

a snapshot per object

the column values as loaded, kept beside the object

the action queue

inserts, updates and deletes waiting for the flush

your objects are the ORM's: proxies and wrappers

a lazy reference is a generated subclass that loads on first touch; a collection is replaced by the ORM's own (PersistentBag) that records adds and removes; enhanced bytecode can mark fields dirty in their setters

- 1

load
objects enter the session, with snapshots
- 2

change
task.setTitle(...): no SQL yet
- 3

flush
before a query, or at commit: compare every managed object with its snapshot
- 4

SQL
UPDATEs for the dirty ones, in the queue's order

```
Doc doc = em.find(Doc.class, 1L);           // managed, with a snapshot
doc.setTitle("User guide");                 // no SQL yet
doc.getSections().remove(1);                // the wrapped list records it
em.createQuery("from Doc").getResultList(); // auto flush: UPDATE, DELETE
tx.commit();                               // flushes whatever is left
```

WHERE THE SNAPSHOT LIVES
inside the session, for every object it manages, until the session closes

HOW A CHANGE IS NOTICED
at flush, by comparing each managed object with its snapshot, or by flags set in enhanced setters

WHEN SQL RUNS
when the session decides: before a query that might see the change (auto flush), or at commit

ACROSS REQUESTS
a detached object must be merged, which reloads it to have a snapshot again

WHAT CAN SURPRISE YOU
a flush you did not ask for; a lazy load after the session closed; an N+1 behind a getter

IN RUST
no runtime proxies: SeaORM makes tracking explicit with ActiveModel's Set and Unchanged values; Diesel has none

Mabat · explicit: two values, compared when you save

```
let before = mabat::load::<Doc>().by_key(1).one(&mut tx).await?;
let mut after = before.clone();           // plain values
after.title = "User guide".into();
after.sections.remove(1);                 // gone
after.sections.swap(0, 1);                // moved
after.sections[0].heading = "Questions".into();
after.sections.push(section(4, "Changes")); // new
mabat::save_changes(&before, &mut after, &mut tx).await?;
```

a comparison the derive generated for Doc
columns and embedded values by PartialEq (a type without it counts as changed) · references by key · owned collections element by element, matched by key · links as lists of keys (MPA-WRITE-8)

```
-- the title changed
UPDATE "doc" SET "title" = $1, "version" = "version" + 1
  WHERE "id" = $2 AND "version" = $3
-- section 2 is gone: deleted with what it owns
DELETE FROM "section" WHERE "id" = ANY($1)
-- section 3: a new heading, and moved to position 0
UPDATE "section" SET "heading" = $1, "position" = $2,
  "version" = "version" + 1 WHERE "id" = $3 AND "version" = $4
-- section 1: moved only, so its position
UPDATE "section" SET "position" = $1, "version" = "version" + 1 ..
-- section 4 is new: saved whole, inserted when no row has its key
INSERT INTO "section" (...) VALUES (...) ON CONFLICT ("id") DO NOTHING
```

one statement per changed row, run now, in a transaction or a savepoint of yours; a value without changes runs none. A stale before matches no version: Error::Conflict.

WHERE THE SNAPSHOT LIVES
in your code: before is an ordinary value you keep, clone, cache, or reload

HOW A CHANGE IS NOTICED
when save_changes is called, by generated code comparing before with after; no proxies, no wrappers, no bytecode

WHEN SQL RUNS
when you call save_changes, in your transaction or a savepoint of it

ACROSS REQUESTS
keep before (or reload it); #[view(version)] makes a stale one fail with Error::Conflict (MPA-WRITE-9)

WHAT CAN SURPRISE YOU
nothing is saved unless you call it; a before that is not what was loaded writes the wrong difference, unless versioned

IN RUST
the structs stay plain: Clone and PartialEq are your choice, and the view is still a view for reading

How an ORM does it. JPA providers such as Hibernate keep a persistence context: an identity map with one managed object per row, a snapshot of each object's state as loaded, and a queue of pending writes. Lazy references are proxies — generated subclasses that load on first touch — and collections are replaced by the provider's own, which record changes. At flush, before a query that might see a change or at commit, each managed object is compared with its snapshot (or, with bytecode enhancement, its setters will have marked it dirty), and the differences become SQL. It is convenient and invisible, and so is when and why SQL runs.

How Mabat does it. Values are plain structs with no session behind them, so there is nothing to intercept a change. The snapshot is `before`, a value you hold. `save_changes` compares it with `after` through a comparison the derive generated: columns and embedded values by `PartialEq` (a type without it counts as changed), to-one foreign keys, owned collection elements matched by key — changed ones updated, new ones saved whole, removed ones deleted with what they own, moved ones of an `index` list given their position — and link tables whose keys differ. Rows that did not change produce no statement [MPA-WRITE-8](#).

What it buys, and costs. SQL runs exactly when you call it, in your transaction or a savepoint of it [MPA-WRITE-1](#); there is no unit of work that flushes later [MPA-NOT-3](#), and no lazy load after a session closed, because there is none [MPA-NOT-4](#). `before` can be kept across requests, cached or reloaded; it must be what the database held, and a `#[view(version)]` column turns a stale one into `Error::Conflict` instead of a wrong write [MPA-WRITE-9](#). The two values must have the same key. In Rust, which has no runtime proxies, the same explicitness is common: `SeaORM`'s `ActiveModel` marks fields `Set` or `Unchanged` by hand.

The same view loads and saves on PostgreSQL, MySQL 8 and SQLite through SQLx 0.9; each enabled database gets its own decoder and encoder, and a load runs on the database of the connection it is given. Where the databases differ, Mabat uses each one's own syntax; the shapes, plans, aliases, checks and errors are the same on all three.

	PostgreSQL	MySQL 8	SQLite
Enabled by	the postgres feature, the default MPA-DB-1	the mysql feature	the sqlite feature
Keys of a batched query	one array parameter: = ANY(\$1)	IN (?, ...), padded to a power of two so statements are reused, at most 1,000 keys a child statement MPA-DB-4	as MySQL
Keys in an override	= ANY(:keys), or \$1 MPA-OVR-4	IN (:keys), expanded to the placeholders	as MySQL
Key types	i16, i32, i64, String, Uuid MPA-VIEW-2	also unsigned integers	also columns without a declared type, read by value
Case-insensitive match	ILIKE MPA-LOAD-5	LOWER(c) LIKE LOWER(?)	LOWER(c) LIKE LOWER(?)
Enum tag columns	any type, such as a PostgreSQL enum, read as text MPA-SUM-1	any type, such as ENUM(...), read as text	any type, read as text
Concurrent loads	Pooled::snapshot or Pooled::read_committed MPA-LOAD-11	Pooled::read_committed; a snapshot does not compile	Pooled::read_committed; a snapshot does not compile
Save: a new row	INSERT ... ON CONFLICT (key) DO UPDATE MPA-WRITE-3	INSERT ... AS new ON DUPLICATE KEY UPDATE	INSERT ... ON CONFLICT (key) DO UPDATE
Versioned insert, if absent	INSERT ... ON CONFLICT (key) DO NOTHING MPA-WRITE-9	INSERT ... SELECT ... WHERE NOT EXISTS, since ON DUPLICATE KEY counts a found row as affected	INSERT ... ON CONFLICT (key) DO NOTHING
Types only some support	#[view(databases = "postgres, mysql")] limits a view to the listed databases, for field types not every enabled database can decode, such as a PostgreSQL array, or rust_decimal::Decimal on SQLite MPA-DB-2 . A registry is built for one database; using it on another fails with Error::WrongBackend MPA-DB-3 .		

One trait, three implementations. mabat-sqlx runs everything through a Backend trait implemented for each database: how rows are fetched and decoded, how keys are read and bound, how statements and their arguments are executed, how column types are described for the checks, and how a snapshot begins. The derive generates a decoder and an encoder for each enabled database, so a view whose field type one database cannot handle fails to compile there, not at run time — unless it opts out of that database.

The same everywhere. The planner and the SQL renderer in mabat-core take a dialect, not a connection: the plan of a view, its query names, its aliases and the order of its queries do not depend on the database. The checks of overrides prepare and describe queries on whichever database the registry is built for, and the tests run on all three databases, including end-to-end suites against the Chinook sample database on each and Pagila on PostgreSQL.

Every capability of Mabat, against the areas of the MPA specification, with the rules that define it. The matrix is drawn from `docs/mpa.json` when this document is built, so it cannot drift from the index; the index cannot drift from the specification, and neither from the crates, because a test fails when the derive accepts an attribute, or the crate has an error variant, that the index does not list.

capability, from docs/mpa.json		CORE 5 rules	DB 5 rules	VIEW 12 rules	SUM 4 rules	LOAD 18 rules	PLAN 5 rules	OVR 8 rules	JSON 7 rules	GQL 4 rules	WRITE 20 rules	SCH 7 rules
typed-views	Declare the shape of nested data with Rust structs deriving View	1		1								
batched-loading	Load a view with one batched query per relationship, never one per row						1					
multiple-databases	PostgreSQL, MySQL 8+ and SQLite, chosen by features and by the connection		1, 2									
embedded-values	Structs stored in columns of the containing view			3, 7								
enums-with-data	Rust enums with data, stored in columns or in a table per variant, decoded strictly				1–4							
json-columns	Columns decoded and written as JSON with serde			6								
to-one-references	References to other views by foreign key, owned, boxed, shared or into a graph			8								
collections	To-many collections, ordered, placed by index, or as maps			9, 10								
many-to-many	Collections through a link table			9								
recursive-views	Trees and chains of parents by depth-limited levels or one WITH RECURSIVE query			8		12	4					
shared-values	Arc<T> values decoded once per entity and shared					13						
graphs	Cyclic data as a Graph of entities with typed references					14						
filters-and-paging	Filters, ordering, paging and counting of the root rows					4–6						
nested-arguments	Filter, order and page the elements of a collection per parent, in one query					9, 10						
streaming	Load many values a batch at a time, as a stream, holding one batch in memory					15–18						
concurrent-loads	Run the queries of each level concurrently on a pool, optionally in one snapshot					11						
sql-overrides	Replace any query of a view with tuned SQL from a file, checked at startup							1–5				
shadow-mode	Run an override next to the generated query and log differences							6				
override-reload	Reload override files without restarting							7				
schema-snapshots	A snapshot of the database's schema, its drift, and views checked against it without a database											1–7
dba-tooling	A manifest of the views and the mabat command line tool: check, explain, scaffold							8				
json-loading	Load views as JSON, whole or a selection of their fields								1, 3–5, 7			
graphql	A GraphQL schema generated from views, each root field one load									1–4		
save-aggregates	Save a value and everything it owns, creating or replacing rows by key										1, 3–6	
save-changes	Save only what changed between two values of an aggregate										8	
optimistic-locking	Version columns that make stale writes fail										9, 10	
save-many	Save many values with statements per table and level, not per row										19	
save-graphs	Save every entity of a graph, ordered by its references, cycles included										14–18	
save-graph-changes	Save only the entities of a graph that were inserted or handed out by get_mut										20	
generated-keys	Keys generated by the database on insert, written back into the value										10, 13	
delete-aggregates	Delete a value and everything it owns										7	

31 capabilities, 95 rules in 11 areas, plus 4 conventions (DOC) and 7 things Mabat 0.1 does not do (NOT). A number is a rule of its column's area: VIEW 9 is MPA-VIEW-9. 25 rules of these areas define details no capability names on its own.

Reading it. A number is a rule of its column's area: under VIEW, 9 is [MPA-VIEW-9](#). A capability's row says where its contract is; a column says which capabilities an area of the specification serves. The areas are the sections of the specification: concepts, databases, declaring views, enums with data, loading, query planning, overrides, JSON and selections, GraphQL, writing, and schema snapshots. Rules that define details — the errors, the decoding of a NULL, the order of children — are in the specification without a row of their own.

Written for tools. `docs/mpa.json` lists every capability, attribute, function, error and diagnostic with the rules that define it, and `11ms.txt` at the root of the repository points assistants to it. Rule identifiers are never reused: a removed rule keeps its number, marked removed [MPA-DOC-2](#). A code review, an issue or an assistant can cite [MPA-WRITE-9](#) and mean one sentence, in one version of the contract.

What is not there. Section 12 of the specification lists what Mabat 0.1 does not do, so tools do not generate code that relies on it: a unit of work, lazy loading, writes through overrides, filters on embedded columns, GraphQL mutations, pipelining, schema generation or migrations — views describe existing tables — and saving only what changed in a graph [MPA-NOT-3](#) to [MPA-NOT-10](#). A rule that is lifted keeps its number, marked removed: keys generated by the database were [MPA-NOT-1](#), and saving graphs [MPA-NOT-2](#).